

Synchronization

Intro

- This week
 - Background
 - The Critical-Section Problem
 - User Library Support for Synchronization
 - Mutex Locks
 - Semaphores
 - Monitors

Synchronization Outline

- This week

- Background
- The Critical-Section Problem
- User Library Support for Synchronization

Next week: Low-level Algorithmic Solutions

- C11 Atomic operations library
- Implementation of locks
 - kernel space
 - user level implementation
- Cache coherence
- Lock “Free” Multithreading
 - memory barriers
- Lock free data structures
 - RCU
- transactions

Next next week

- Review and summary of synchronization

```
void *producer(void *data){
    while (1)    {
        /* produce an item in next produced
        */

        while (count == BUFFER_SIZE)
            ; /* do nothing */

        buffer[in] = produced;
        in = (in + 1) % BUFFER_SIZE;
        count++;
    }
}
```

```
void *consumer(void *data){
    while (1){

        while (count == 0)
            ; /* do nothing */

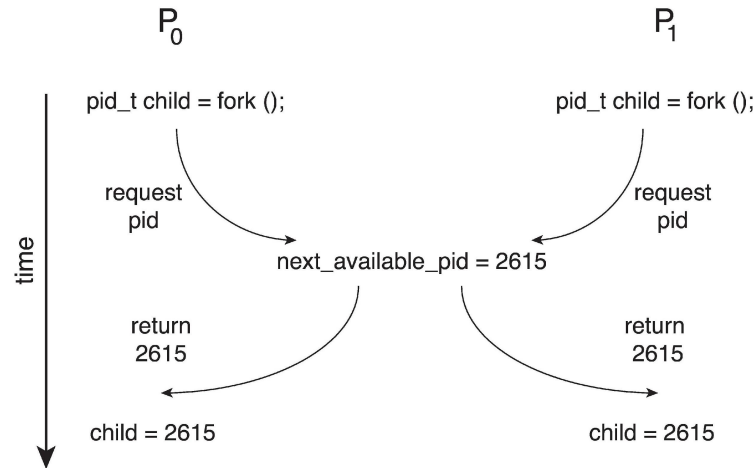
        consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        count--;

        /* consume the item in next
        consumed */
    }
}
```

// UNSYNCHRONIZED - THIS CODE HAS A RACE CONDITION

Race Condition Example in Kernel

- Processes P_0 and P_1 are creating child processes using the `fork()` system call
- Race condition on kernel variable `next_available_pid` which represents the next available process identifier (pid)



- Unless there is a mechanism to prevent P_0 and P_1 from accessing the variable `next_available_pid` the same pid could be assigned to two different processes!

Explicit Example in User program

```
#include <stdio.h>
#include <pthread.h>
```

```
int sum = 0; //shared
```

```
void *countgold(void *param) {
    int i; //local to each thread
    for (i = 0; i < 10000000; i++) {
        sum += 1;
    }
    return NULL;
}
```

```
int main() {
    pthread_t tid1, tid2;
    pthread_create(&tid1, NULL, countgold, NULL);
    pthread_create(&tid2, NULL, countgold, NULL);

    //Wait for both threads to finish:
    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);

    printf("ARRRRG sum is %d\n", sum);
    return 0;
}
```

```
$ gcc main.c -pthread
$ ./a.out
??????????
```

When multithreading gets “interesting”!

```
#include <stdio.h>
#include <pthread.h>
```

```
int sum = 0; //shared
```

```
void *countgold(void *param) {
    int i; //local to each thread
    for (i = 0; i < 10000000; i++) {
        sum += 1;
    }
    return NULL;
}
```

```
int main() {
    pthread_t tid1, tid2;
    pthread_create(&tid1, NULL, countgold, NULL);
    pthread_create(&tid2, NULL, countgold, NULL);

    //Wait for both threads to finish
    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);

    printf("ARRRRRG sum is %d\n", sum);
    return 0;
}
```

```
$ gcc main.c -pthread
$ ./a.out
ARRRRRG sum is 10214142
$ ./a.out
ARRRRRG sum is 11798888
$ ./a.out
ARRRRRG sum is 10576703
$ ./a.out
ARRRRRG sum is 10159994
```

There is a race condition if the end result may change based on the execution order.

Mutual exclusion

From previous example

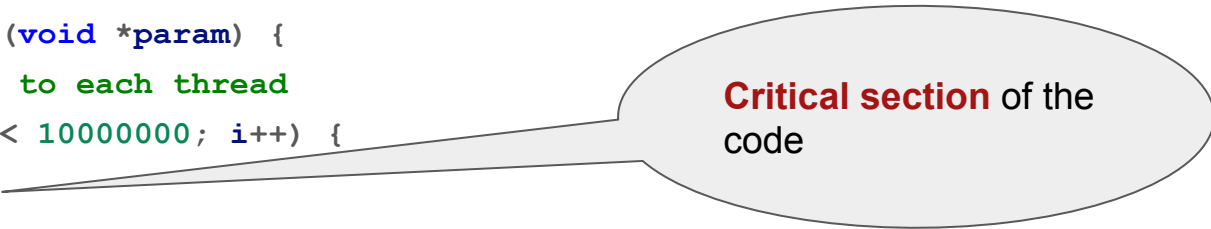
```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
int sum = 0; //shared
```

```
void *countgold(void *param) {  
    int i; //local to each thread  
    for (i = 0; i < 10000000; i++) {  
        sum += 1;  
    }  
    return NULL;  
}
```

We do not want two or more threads to update **sum** at the same time.



Critical section of the
code

Mutual exclusion

Mutual exclusion is a property that says

“no two or more processes (threads) can be in their critical section at the same time at any given point of time”.

Critical Section

- General structure of process P_i

```
while (true) {  
    entry section  
    critical section  
    exit section  
    remainder section  
}
```

Requirements for solution to critical-section problem

1. Mutual Exclusion

- If process P_i is executing in its critical section, then no other processes can be executing in their critical sections

2. Progress

- If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the process that will enter the critical section next cannot be postponed indefinitely

3. Bounded Waiting

- A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted

Assumptions:

- Assume that each process executes at a nonzero speed
- No assumption concerning **relative speed** of the n processes

Hardware solutions:

Interrupt-based solution

- Entry section: disable interrupts
- Exit section: enable interrupts

```
while (true) {
```

```
    entry section
```

```
    critical section
```

```
    exit section
```

```
    remainder section
```

```
}
```

- Will this solve the problem?
 - What if the critical section-code runs for an hour?
 - Can some processes starve
 - never enter their critical section.
 - What if there are two CPUs?

Software-user library solutions

Thread packages typically provide
mutex variables

- acquire-release
- lock-unlock

semaphores (through system calls)

Pthread (POSIX Threads) Libraries for User Apps

Thread packages typically provide mutexes:

```
void mutex_init (mutex_t *m, ...);
```

```
void mutex_lock (mutex_t *m);
```

```
int mutex_trylock (mutex_t *m);
```

```
void mutex_unlock (mutex_t *m);
```

- Only one thread acquires m at a time, others wait

Mutual exclusion

Mutual exclusion is a property that says

“no two or more processes (threads) can be in their critical section at the same time at any given point of time”.

Thread synchronization

Mutex ("mutual exclusion")

- Protect accesses to shared variables at the same time

Condition Variables

- Inform another thread about the change in the state of the shared variable
 - synchronize based upon the actual value of the variable

Mutex variables

To achieve mutual exclusion we use mutex variable

It ensures that only one thread at a time can access a global variable

In POSIX: `pthread_mutex_t`,

similar lock concepts in different languages

```
/* global variable should be used for  
static mutexes*/
```

```
pthread_mutex_t m;
```

```
m = PTHREAD_MUTEX_INITIALIZER;
```

```
/* start of Critical Section */
```

```
pthread_mutex_lock(&m);
```

```
// Critical section
```

```
/* end of Critical Section */
```

```
pthread_mutex_unlock(&m);
```


Mutexes

```
//m1 is a mutex variable  
mutex_lock(m1); //acquire lock  
critical_section  
mutex_unlock(m1); //release lock
```

- You can consider lock as “a mic among participants that controls who has the right to speak”,
 - That is whoever has the mic (m1) has the right to speak
 - in our case do ops on the memory shared among the participants.

Mutex variables

- A mutex can either be allocated as a static (global) variable
- or be created dynamically at run time

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;  
pthread_mutex_lock(&m);  
// Critical section  
pthread_mutex_unlock(&m);
```

```
pthread_mutex_t *lock = malloc(sizeof(pthread_mutex_t));  
pthread_mutex_init(lock, NULL);  
//critical section  
pthread_mutex_destroy(lock);  
free(lock);
```

Example Sum

```
/* Create a mutex this ready to be locked!*/
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
int sum = 0;
void *countgold(void *param) {

    pthread_mutex_lock(&m);
    /* Other threads that call lock will have to wait until we call unlock */
    for (int i = 0; i < 10000000; i++) {
        sum += 1; /*->critical section*/
    }
    pthread_mutex_unlock(&m);

    return NULL;
}
```

- Same thread that locks the mutex must unlock it
- Critical section is 'sum += 1'

Which one runs faster?

```
void *countgold(void *param) {  
    int i;  
    for (i = 0; i < 100000000; i++) {  
        pthread_mutex_lock(&m);  
        sum += 1;  
        pthread_mutex_unlock(&m);  
    }  
    return NULL;  
}
```

This runs slower because we lock and unlock the mutex a million times, which is expensive - at least compared with incrementing a variable.

```
void *countgold(void *param) {  
    int i, local = 0;  
    for (i = 0; i < 100000000; i++) {  
        local += 1;  
    }  
  
    pthread_mutex_lock(&m);  
    sum += local;  
    pthread_mutex_unlock(&m);  
  
    return NULL;  
}
```

Example: sum of elements in array

Write a concurrent program to find the sum of elements in array

- Define a global sum
- Each thread sums elements between start and end
 - Start & end are computed from thread index
- Adds its resulting sum to the global sum

Some gotchas!

```
int a;

pthread_mutex_t m1 = PTHREAD_MUTEX_INITIALIZER,
                 m2 = PTHREAD_MUTEX_INITIALIZER;

// later
// Thread 1

pthread_mutex_lock(&m1);

a++;

pthread_mutex_unlock(&m1);

// Thread 2

pthread_mutex_lock(&m2);

a++;

pthread_mutex_unlock(&m2);
```

This code creates a mutex
that does effectively
nothing!

Condition variables: Overview

mutexes

- implement synchronization by controlling thread access to data,
- prevent multiple threads accessing the shared variable at the same time.
 - Threads must be participating

condition variables

- allow threads to synchronize based upon the actual value of data.
 - one thread informs other threads about changes in the state of a shared variable (or other shared resource)
 - and allows other threads to wait (block) for such notification.
- allow a set of threads to sleep until woken up.
- A condition variable is always used in conjunction with a mutex lock

Monitor

a monitor is

- a synchronization construct
- “Pattern”

A monitor consists of

- a mutex (lock)
 - It uses the **lock** to protect a critical section.
- and at least **one condition variable**.
 - allows them to wait for the state to change

It provides a mechanism for threads to temporarily give up exclusive access in order to wait for some condition to be met

A basic monitor construct

```
lock(m);                // Acquire this monitor's lock.

/*... Critical section ...*/

while (!p) {            // While the condition that we are waiting for is not true
    wait(cv, m);         // Wait on this monitor's lock and condition variable.
}

/*... Critical section ...*/

signal(cv2);             // Or: broadcast(cv2);
                        // cv2 might be the same as cv or different.

unlock(m);              // Release this monitor's lock.
```

Monitor for waiting two different conditions

```
//1st thread
mutex_lock(m1);

/*critical section entry*/
while(need_to_wait_1 ){
    cond_wait(c1, m1);
}

/*critical section exit*/
cond_signal(c2) //or broadcast
mutex_unlock(m1);
```

```
// 2nd thread
mutex_lock(m1);

/*critical section entry*/
while(need_to_wait_2){
    cond_wait(c2, m1);
};

/*critical section exit*/
cond_signal(c1) //or broadcast
mutex_unlock(m1);
```

- m1: mutex lock to control the access to the critical section.
- c1: condition variable is for one condition
- c2: condition variable is for another condition

Creating condition variables

Condition variables must be declared with type `pthread_cond_t`, and must be initialized before they can be used. There are two ways to initialize a condition variable:

Statically,

```
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
```

Dynamically,

```
int pthread_cond_init(pthread_cond_t *cond, pthread_condattr_t * attr);
```

- The ID of the created condition variable is returned to the calling thread through the condition parameter.
 - This method permits setting condition variable object attributes, attr.

```
int pthread_cond_destroy(pthread_cond_t *cond);
```

Signaling and waiting on condition variables

```
#include <pthread.h>

int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);

int pthread_cond_signal(pthread_cond_t *cond); /*only one waiting thread is woken*/

int pthread_cond_broadcast(pthread_cond_t *cond); /*all waiting threads are woken*/
```

All return 0 on success, or a positive error number on error

Condition variables are always used with a mutex lock.

Before calling wait, the mutex lock must be locked and wait must be wrapped with a loop.

```
pthread_mutex_lock();  
    while(condition_is_false)  
        pthread_cond_wait();  
pthread_mutex_unlock();
```

A **while loop** is used because the condition could change after `pthread_cond_wait()`.

- Wait wakes the thread but the state of the buffer might change until the thread gets to really run.

producer-consumer example

```
pthread_mutex_t mtx =
    PTHREAD_MUTEX_INITIALIZER;
int avail = 0;
void* producer(void * p) {
    pthread_mutex_lock(&mtx);
    avail++;
    pthread_mutex_unlock(&mtx);
}

int main() {
    for (;;) {
        pthread_mutex_lock(&mtx);
        while (avail > 0) {
            avail--;
        }
        pthread_mutex_unlock(&mtx);
    }
}
```

The above code works, but it wastes CPU time, because the main thread continually loops, checking the state of the variable `avail`.

producer-consumer example with cond variable

```
pthread_mutex_t mtx =  
    PTHREAD_MUTEX_INITIALIZER;  
int avail = 0;  
void *producer(void *p) {  
    pthread_mutex_lock(&mtx);  
    avail++;  
    pthread_cond_signal(&cv);  
    pthread_mutex_unlock(&mtx);  
}
```

```
void *consumer(void *p) {  
    for (;;) {  
        pthread_mutex_lock(&mtx);  
        while (avail == 0) {  
            pthread_cond_wait(&cv, &mtx);  
        }  
        /*consume*/  
        while (avail > 0) {  
            avail--;  
        }  
        pthread_mutex_unlock(&mtx);  
    }  
}
```

A condition variable allows a thread to sleep (wait) until another thread notifies (signals) it that it must do something

Details of wait

```
pthread_cond_wait(&cv, &mtx)
```

- unlocks the mutex
- and begins waiting for the condition variable to be signalled.

```
pthread_mutex_lock(&mtx);  
while (avail == 0) {  
    pthread_cond_wait(&cv, &mtx);  
}
```

- before invoking wait(), you must have ownership of **mtx**
- pthread_cond_wait() returns with the mutex **locked! You must unlock at the end!**

```
pthread_mutex_unlock(&mtx);
```


If there is no-space, to prevent overflow:

★ You can also add condition to producer.

The main design is the following:

```
//1st thread
mutex_lock(m1);
/*critical section entry*/
while(need_to_wait_1){
    cond_wait(c1, m1);
}

/*critical section exit*/
cond_signal(c2) //or broadcast
mutex_unlock(m1);
```

```
// 2nd thread
mutex_lock(m1);
/*critical section entry*/
while(need_to_wait_2){
    cond_wait(c2, m1);
};

/*critical section exit*/
cond_signal(c1) //or broadcast
mutex_unlock(m1);
```

Producer-consumer with overflow control

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t notempty = PTHREAD_COND_INITIALIZER;
pthread_cond_t notfull = PTHREAD_COND_INITIALIZER;
int count = 0;
int buffer[BUFFER_SIZE];
```

```
void *producer(void *ptr) {

    pthread_mutex_lock(&m);
    while (count >= BUFFER_SIZE)
        pthread_cond_wait(&notfull, &m);

    /*produce 1 item*/
    printf("producing item:%d", count);
    buffer[count] = rand();
    count = count + 1;

    pthread_cond_signal(&notempty);
    pthread_mutex_unlock(&m);
}
```

```
void *consumer(void *ptr) {
    // for(;;){
    pthread_mutex_lock(&m);
    while (count == 0)
        pthread_cond_wait(&notempty, &m);

    /*consume 1 item*/
    printf("consumed item %d - %d\n",
           count, buffer[count]);
    count = count - 1;

    pthread_cond_signal(&notfull);
    pthread_mutex_unlock(&m);
}
```

What if we have more than one shared resources?

- use many mutexes?

What about processes?

Semaphores: Overview

```
sem_t s;  
sem_init(&s, 0, 10); //WE HAVE 10 available resources (say 10-chairs)  
  
//a thread that wants to occupy a chair  
sem_wait(&s); //down the value of s by 1  
...  
  
//After done with the chair  
sem_post(&s); //up the value of s by 1
```

- If the return value of `sem_wait` is negative the thread waits as done in the mutex locks.
 - This happens when the value of `s` before `sem_wait` is 0.
- `sem_wait` and `sem_post` can be called from different processes/threads.

POSIX Semaphores

```
int sem_init(sem_t *sem,  
             int pshared,  
             unsigned int value);
```

initializes the unnamed semaphore at the address pointed to by `sem`.

value

- specifies **the initial value** for `sem`

pshared: indicates whether between process or threads

- 0 between threads
 - Should be visible to all threads,
 - Global variable
- Non-zero between processes

allow **processes** and **threads** to
synchronize access to shared resources

A semaphore is a type of lockable object.

```
sem_t s;
```

```
sem_init(&s, 0, 10); /*returns -1 on error*/
```

Decrementing the semaphore value

```
sem_t s;
```

```
sem_init(&s, 0, 10); /*returns -1 on  
error*/
```

decrementing the value of semaphore: You take ownership of a semaphore with a *wait* operation (abstractly called *P* from Dijkstra's paper),

```
sem_wait(&s);
```

- Could do this 10 times without blocking

POSIX Semaphores

incrementing the value of semaphore:

You release ownership with a *signal* operation, a *post* operation, or abstractly called V.

`sem_post(&s);`

- **Announce that we've finished**
- **And one more resource item is available;**
- **increment the value of semaphore**

To destroy semaphore

- release resources of the semaphore

`sem_destroy(&s);`

Binary semaphore vs mutex

A binary semaphore is a semaphore with a maximum count of 1.

```
sem_t s;
```

```
sem_init(&s, 0, 1);
```

- A mutex is a semaphore that always **waits** before it **posts**.
- A binary semaphore can be used as a mutex by requiring that a thread only signals the semaphore (to unlock the mutex) if it was the thread that last successfully waited on it (when it locked the mutex).

- mutex ownership is tied to a thread, and only the thread that acquired the lock on a mutex can release it
- the semaphore doesn't care if the "wrong" thread signals the semaphore.
- `sem_wait` and `sem_post` can be called from different threads!

Semaphore vs mutex

```
1. // Thread 1
2. sem_wait(&s);
3. // Critical Section
4. sem_post(&s);

5. // Thread 2
6. // Some threads want to see the world burn
7. sem_post(&s);

8. // Thread 3
9. sem_wait(&s);
10. // Not thread-safe!
11. sem_post(&s);
```

```
1. // Thread 1
2. mutex_lock(&s);
3. // Critical Section
4. mutex_unlock(&s);

5. // Thread 2
6. // Failed!
7. mutex_unlock(&s);

8. // Thread 3
9. mutex_lock(&s);
10. // Now it's thread-safe
11. mutex_unlock(&s);
```

Making Data Structures Thread-Safe

```
int count = 0;
double values[STACK_SIZE];
void push(double v) {
    values[count++] = v;
}
double pop() {
    return values[--count];
}
int is_empty() {
    return count == 0;
}
```

This implementation of the stack is thread-unsafe because if two threads call push or pop at the same time then the results or the stack can be inconsistent.

For example, imagine if two threads call pop at the same time then both threads may read the same value, both may read the original count value.

Version-2 of stack

```
#define STACK_SIZE 20

int count;

double values[STACK_SIZE];

pthread_mutex_t m1 =
PTHREAD_MUTEX_INITIALIZER;

pthread_mutex_t m2 =
PTHREAD_MUTEX_INITIALIZER;

void push(double v) {
    pthread_mutex_lock(&m1);
    values[count++] = v;
    pthread_mutex_unlock(&m1);
}
```

```
double pop() {
    pthread_mutex_lock(&m2);
    double v = values[--count];
    pthread_mutex_unlock(&m2);
    return v;
}

int is_empty() {
    pthread_mutex_lock(&m1);
    return count == 0;
    pthread_mutex_unlock(&m1);
}
```

This version contains 2 errors!
What are the errors?

Version 3

```
// Another attempt to a thread-safe stack
#define STACK_SIZE 20

int count;

double values[STACK_SIZE];

pthread_mutex_t m1 =
PTHREAD_MUTEX_INITIALIZER;

void push(double v) { /*may be overflow*/
    pthread_mutex_lock(&m1);
    values[count++] = v;
    pthread_mutex_unlock(&m1); }
```

```
double pop() { /*may be underflow*/
    pthread_mutex_lock(&m1);
    double v = values[--count];
    pthread_mutex_unlock(&m1);
    return v;
}

int is_empty() {
    pthread_mutex_lock(&m1);
    int r = count == 0;
    pthread_mutex_unlock(&m1);
    return r; /*may be out of date*/
}
```

- `is_empty` is thread-safe but its result may already be out-of-date. This is usually why in thread-safe data structures, functions that return sizes are removed or deprecated.
- There is no protection against underflow (popping on an empty stack) or overflow (pushing onto an already-full stack)

A general stack

A more general-purpose version might include the mutex as part of the memory structure and use `pthread_mutex_init` to initialize the mutex.

```
typedef struct stack {
    int count;
    pthread_mutex_t m;
    double *values;
} stack_t;

stack_t* stack_create(int capacity) {
    stack_t *result = malloc(sizeof(stack_t));
    result->count = 0;
    result->values = malloc(sizeof(double) * capacity);
    pthread_mutex_init(&result->m, NULL);
    return result;
}

void stack_destroy(stack_t *s) {
    free(s->values);
    pthread_mutex_destroy(&s->m);
    free(s);
}
```

```

void push(stack_t *s, double v){
    pthread_mutex_lock(&s->m);
    s->values[(s->count)++] = v;
    pthread_mutex_unlock(&s->m);
}

double pop(stack_t *s){
    pthread_mutex_lock(&s->m);
    double v = s->values[--(s->count)];
    pthread_mutex_unlock(&s->m);
    return v;
}

int is_empty(stack_t *s){
    pthread_mutex_lock(&s->m);
    int result = s->count == 0;
    pthread_mutex_unlock(&s->m);
    return result;
}

```

```

int main(){
    stack_t *s1 = stack_create(10 /* Max
capacity*/);
    stack_t *s2 = stack_create(10);
    push(s1, 3.141);
    push(s2, pop(s1));
    stack_destroy(s2);
    stack_destroy(s1);
}

```

We still need to control overflow/underflow in pop and push

This can be done by using semaphores

With semaphores

```
#define SPACES 10

pthread_mutex_t m=
PTHREAD_MUTEX_INITIALIZER;

int count = 0;

double values[SPACES];

sem_t sitems, sremain;

void init() {
    sem_init(&sitems, 0, 0);
    sem_init(&sremains, 0, SPACES);
}
```

```
double pop() {
    /* Wait until there's
       at least one item*/
    sem_wait(&sitems);
    pthread_mutex_lock(&m);
    /* CRITICAL SECTION */
    double v = values[--count];
    pthread_mutex_unlock(&m);

    /*there's at least one space*/
    sem_post(&sremain);
    return v;
}
```

```
void push(double v) {  
    /* Wait until there's at least one space*/  
    sem_wait(&sremain);  
    pthread_mutex_lock(&m);  
    /* CRITICAL SECTION */  
    values[count++] = v;  
    pthread_mutex_unlock(&m);  
    sem_post(&sitems); /* Hey world, there's at least one item*/  
}
```


Exercise

How to modify the previous stack example for processes?

- Instead of `mutex_lock`, you need to use binary semaphore..

Implementing with condition variable: WRONG!

```
void push(stack_t *s, double v) {  
    pthread_mutex_lock(&s->m);  
    if(s->count == 0) pthread_cond_wait(&s->cv, &s->m);  
    s->values[(s->count)++] = v;  
    pthread_mutex_unlock(&s->m);  
}
```

```
double pop(stack_t *s) {  
    pthread_mutex_lock(&s->m);  
    if(s->count == 0) pthread_cond_wait(&s->cv, &s->m);  
    double v = s->values[--(s->count)];  
    pthread_mutex_unlock(&s->m);  
    return v;  
}
```

Errors?

1. The first one is a simple one. In push, our check should be against the total capacity, not zero.
2. We only have if statement checks. wait() could spuriously wake up
3. We never signal any of the threads! Threads could get stuck waiting indefinitely.

Implementing with condition variable

```
void push(stack_t *s, double v) {
    pthread_mutex_lock(&s->m);
    while(s->count == capacity) pthread_cond_wait(&s->cv, &s->m);
    s->values[(s->count)++] = v;
    pthread_cond_signal(&s->cv);
    pthread_mutex_unlock(&s->m);
}

double pop(stack_t *s) {
    pthread_mutex_lock(&s->m);
    while(s->count == 0) pthread_cond_wait(&s->cv, &s->m);
    double v = s->values[--(s->count)];
    pthread_cond_broadcast(&s->cv);
    pthread_mutex_unlock(&s->m);
    return v;
}
```

- Signal wakes any thread
- Which might be another push thread

better to implement with two condition variable

See producer-consumer example

Implementing with 2 condition variables

```
void push(stack_t *s, double v) {  
    pthread_mutex_lock(&s->m);  
    while(s->count == capacity) pthread_cond_wait(&s->cv1, &s->m);  
    s->values[(s->count)++] = v;  
    pthread_cond_broadcast(&s->cv2);  
    pthread_mutex_unlock(&s->m);  
  
}  
  
double pop(stack_t *s) {  
    pthread_mutex_lock(&s->m);  
    while(s->count == 0) pthread_cond_wait(&s->cv2, &s->m);  
    double v = s->values[--(s->count)];  
    pthread_cond_broadcast(&s->cv1);  
    pthread_mutex_unlock(&s->m);  
    return v;  
}
```

Synchronization & deadlock

examples

- Reader-writer problem
- Review of producer-consumer problem
- Ring buffer (queue) implementation with condition variables & semaphores
- Deadlock, livelock
- Dining philosopher problem & different solutions

Reader-writer problem

Readers: only reads the data and does not update/change the data

Writers: both reads and writes the data

- allow multiple readers to access the data at the same time when there is no writing.
- To avoid data corruption, only one thread at a time may modify (write) the data structure and no readers may be reading at that time.

Problem: how can we efficiently synchronize multiple readers and writers such that multiple readers can read together, but a writer gets exclusive access?

First attempt with mutex

```
void read() {  
    lock(&m)  
    // do read stuff  
    unlock(&m)  
}
```

```
void write() {  
    lock(&m)  
    // do write stuff  
    unlock(&m)  
}
```

- No data corruption!
- But, only one reader can access the data.
Readers wait for other readers

Second attempt

```
void read() {  
    while(writing) { /*spin*/}  
  
    reading = 1  
    // do read stuff  
    reading = 0  
}  
  
void write() {  
    while(reading || writing) { /*spin*/}  
  
    writing = 1  
    // do write stuff  
    writing = 0  
}
```

- Race condition: multiple readers and the writers at the same time

With condition variables

Condition ok to read

Readers can access/read when no writers

Condition ok to write

Writers can access/update/write when no readers or writers

- ★ Only one thread should manipulate state variables (reading, writing) at a time.
 - Critical section

Reader

```
read() {  
    lock(&m)  
    while (writing)  
        cond_wait(&cv, &m)  
    reading++;  
    /* Read here! */  
    reading--  
    cond_signal(&cv)  
    unlock(&m)  
}
```

`pthread_cond_wait` performs 3 actions.

- ◆ it atomically unlocks the mutex
- ◆ and then sleeps
 - (until it is woken by `pthread_cond_signal` or `pthread_cond_broadcast`).
- ◆ the awoken thread must re-acquire the mutex lock before returning.

→ Thus only one thread can actually be running inside the critical section.

Solution to Reader-Writer problem?

- if any writers are writing, a reader will enter the `cond_wait`.
 - ◆ OK!
- Only one reader can read!
 - ◆ because readers did not unlock the mutex.

A better version for multiple readers

```
read() {  
    lock(&m)  
    while (writing)  
        cond_wait(&cv, &m)  
    reading++;  
    unlock(&m)  
  
    /* Read here! */  
  
    lock(&m)  
    reading--  
    cond_signal(&cv)  
    unlock(&m)  
}
```

Solution to Reader-Writer problem?

- if any writers are writing, a reader will enter the `cond_wait`.
 - ◆ OK!
- only one thread can be executing code inside the **critical section** at any time!
- Readers can read at the same time

Does this mean that a writer and reader could read and write at the same time?

- ◆ No!
 - **cond_wait** requires the thread re-acquire the mutex lock before returning.

Writer

```
write() {  
    lock(&m);  
    while (reading || writing)  
        cond_wait(&cv, &m);  
    writing++;  
  
    /* Write here! */  
  
    writing--;  
    cond_signal(&cv); //or cond_broadcast  
    unlock(&m);  
}
```

Explanation

- This uses `pthread_cond_signal`. This will only wake up one thread.
 - If many readers are waiting for the writer to complete, only one sleeping reader will be awoken from their slumber.
- The reader and writer should use `cond_broadcast` so that all threads should wake up and check their while-loop condition.

Problem with this solution

- Starvation: If readers are constantly arriving then a writer will never be able to proceed (**the 'reading' count never reduces to zero**).

Last attempt

implement a bounded-wait for the writer.

- If a writer arrives they will still need to wait for existing readers
- however future readers must be placed in a “holding pen” and wait for the writer to finish.

The plan is when a writer arrives, and before waiting for current readers to finish, register our intent to write by incrementing a counter ‘writer’

Final writer solution

```
writer() {  
    lock(&m)  
    writers++  
    while (reading || writing)  
        cond_wait(&cv, &m)  
    writing++  
    unlock(&m)  
    // perform writing here  
    lock(&m)  
    writing--  
    writers--  
    cond_broadcast(&cv)  
    unlock(&m)  
}
```

- Readers wait for the writers

final reader solution

```
read() {  
    lock(&m)  
    /*check if any writers waiting*/  
    while(writers)  
        cond_wait(&cv,&m)  
    /* No need to wait while(writing here) because it only exits the above loop when writing is zero*/  
    while(writing)  
        cond_wait(&cv,&m)  
    reading++  
    unlock(&m)  
    /* Read here! */  
    lock(&m)  
    reading--  
    cond_signal(&cv)  
    unlock(&m)  
}
```

- Readers wait for the writers
 - Writers have higher priority

Review: consumer-producer problem

- a finite-size buffer
- and two classes of threads, producers and consumers,
 - producers: put items into the buffer
 - consumers: take items out of the buffer
- A producer must wait until the buffer has space before it can put something in,
- and a consumer must wait until something is in the buffer before it can take something out.

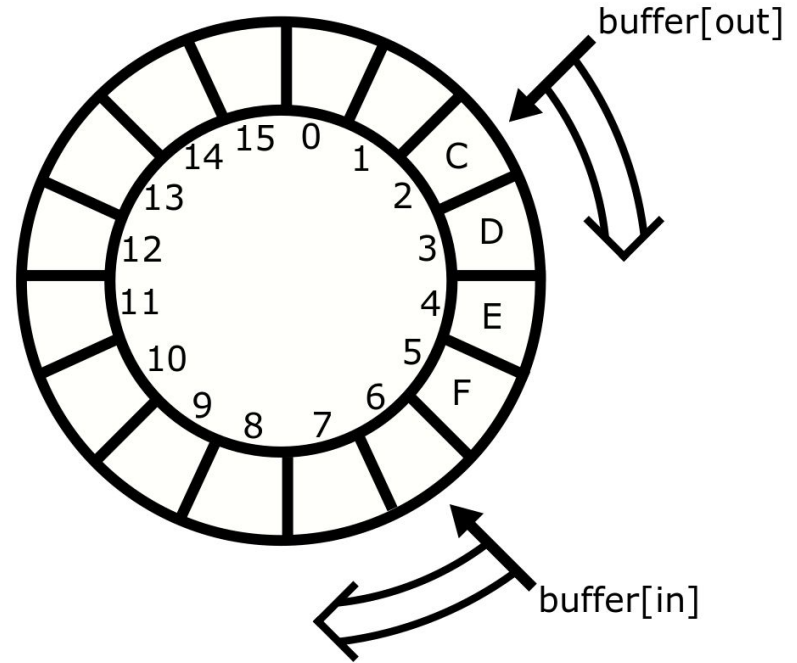
A **condition variable** represents a queue of threads waiting for some condition to be signaled.

Ring buffer

- A ring buffer is a simple, usually fixed-sized, storage mechanism
 - where contiguous memory is treated as if it is circular,
 - and two index counters keep track of the current beginning and end of the queue.
- As an array

```
void *buffer[16];
```

```
unsigned int in = 0, out = 0;
```



Ring buffer queue operations

```
void *buffer[16];  
unsigned int in = 0, out = 0;
```

```
void enqueue(void *value) {  
    buffer[in] = value;  
    in++; /* Advance the index for next time */  
    if (in == 16) in = 0; /* Wrap around! */  
}
```

```
void *dequeue() {  
    void *result = buffer[out];  
    out++;  
    if (out == 16) out = 0;  
    return result;  
}
```

- No control for buffer overflow/underflow!
- `if (in == 16) in = 0;`
 - Using `(in % 16)` may cause problems if `in` is too big
 - `b[ (in++) & (N-1) ] = value;`

```

#define N 16
void *b[N]
int in = 0, out = 0;
p_m_t lock; sem_t s1,s2;
void init() {
    p_m_init(&lock, NULL);
    sem_init(&s1, 0, 16);
    sem_init(&s2, 0, 0);
}
enqueue(void *value) {
    p_m_lock(&lock)
    sem_wait( &s1 )
    b[ (in++) & (N-1) ] = value
    sem_post(&s1)
    p_m_unlock(&lock)
}

```

Multithread solution: **incorrect** attempt

- The enqueue method waits and posts on the same semaphore (s1) and similarly with enqueue and (s2)
 - we decrement the value and then immediately increment the value,
 - so by the end of the function the **semaphore value is unchanged!**
- The initial value of s1 is 16,
 - the semaphore will never be reduced to zero
 - enqueue will not block if the ring buffer is full
 - so overflow is possible.

```

void init() {
    p_m_init(&lock, NULL);
    sem_init(&s1, 0, 16);
    sem_init(&s2, 0, 0);
}

void *dequeue(){
    p_m_lock(&lock)
    sem_wait(&s2)
    void *result = b[(out++) & (N-1) ]
    sem_post(&s2)
    p_m_unlock(&lock)
    return result
}

```

- The initial value of s2 is zero, so calls to dequeue will always block and never return!
- The order of mutex lock and sem_wait will need to be swapped; however, this example is so broken that this bug has no effect!

```

void init() {
    sem_init(&s1,0,16);  sem_init(&s2,0,0);
}

enqueue(void *value){
    sem_wait(&s2);
    p_m_lock(&lock);
    b[ (in++) & (N-1) ] = value;
    p_m_unlock(&lock);
    sem_post(&s1);
}

void *dequeue(){
    sem_wait(&s1);
    p_m_lock(&lock)
    void *result = b[(out++) & (N-1)];
    p_m_unlock(&lock);
    sem_post(&s2);
    return result;
}

```

Another **incorrect** attempt

- The initial value of s2 is 0. **Enqueue will block on the first call** to sem_wait even though the buffer is empty!
- The initial value of s1 is 16. Dequeue **will not block on the first call** to sem_wait even though the buffer is empty - Underflow! The dequeue method will return invalid data.
- The code does not satisfy Mutual Exclusion. Two threads can modify in or out at the same time! The code appears to use mutex lock. Unfortunately, the lock was never initialized with pthread_mutex_init() or PTHREAD_MUTEX_INITIALIZER - so the lock may not work (pthread_mutex_lock may simply do nothing)

Correct implementation: enqueue

```
#include <pthread.h>
#include <semaphore.h>
#define N (16) // N must be 2^i
void *b[N]
int in = 0, out = 0
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
sem_t countsem, spacesem;
void init() {
    sem_init(&countsem, 0, 0);
    sem_init(&spacesem, 0, 16);
}
```

```
enqueue(void *value){
    /* wait if there is no space left:*/
    sem_wait( &spacesem )

    pthread_mutex_lock(&lock)
    b[ (in++) & (N-1) ] = value
    pthread_mutex_unlock(&lock)

    /* increment the count of the number of items*/
    sem_post(&countsem)
}
```

Correct implementation: dequeue

```
#include <pthread.h>
#include <semaphore.h>
#define N (16) // N must be 2^i
void *b[N]
int in = 0, out = 0
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
sem_t countsem, spacesem;
void init() {
    sem_init(&countsem, 0, 0);
    sem_init(&spacesem, 0, 16);
}
```

```
void *dequeue(){
    /* Wait if there are no items in the buffer*/
    sem_wait(&countsem);

    pthread_mutex_lock(&lock);
    void *result = b[(out++) & (N-1)];
    pthread_mutex_unlock(&lock);

    /* Increment the count of the number of spaces*/
    sem_post(&spacesem);
    return result
}
```

Exercises

- What would happen if the order of **sem_wait** and **pthread_mutex_lock** calls were swapped?
- What would happen if the order of **pthread_mutex_unlock** and **sem_post** calls were swapped?

```
void *dequeue(){  
    /* Wait if there are no items in the buffer*/  
    sem_wait(&countsem);  
    p_m_lock(&lock);  
    void *result = b[(out++) & (N-1)];  
    p_m_unlock(&lock);  
    /* Increment the count of the number of  
spaces*/  
    sem_post(&spacesem);  
    return result  
}
```


Ring buffer as a consumer-producer with cvs

```
typedef struct {  
    char buf[BSIZE];  
    int occupied;  
    int in;           /*index of in*/  
    int out;          /*index of out*/  
    pthread_mutex_t mutex;  
    pthread_cond_t more; /*there is item*/  
    pthread_cond_t less;  /*there is room*/  
} buffer_t;
```

```
void enqueue(buffer_t *b, char item){  
    pthread_mutex_lock(&b->mutex);  
    while (b->occupied >= BSIZE)  
        pthread_cond_wait(&b->less, &b->mutex);  
    assert(b->occupied < BSIZE);  
    b->buf[b->in++] = item;  
    b->in &= BSIZE-1;  
    b->occupied++;  
    pthread_cond_signal(&b->more);  
    pthread_mutex_unlock(&b->mutex);  
}
```

```
char dequeue(buffer_t *b){
    char item;
    pthread_mutex_lock(&b->mutex);
    while(b->occupied <= 0){
        pthread_cond_wait(&b->more, &b->mutex);
    }
    item = b->buf[b->out++];
    b->out &= BSIZE-1;
    b->occupied--;
    pthread_cond_signal(&b->less);
    pthread_mutex_unlock(&b->mutex);
    return(item);
}
```

Dequeue is a consumer

Enqueue is a producer

Deadlock

Deadlock describes a situation where two or more threads are blocked forever, waiting for each other.

Example

- a. Thread 1 holds lock A and requests lock B
- b. Thread 2 holds lock B and requests lock A

Deadlock occurs when

- Threads that are already holding locks request new locks
- The requests for new locks are made concurrently
- Two or more threads form a circular chain in which each thread waits for a lock which is held by the next thread in the chain

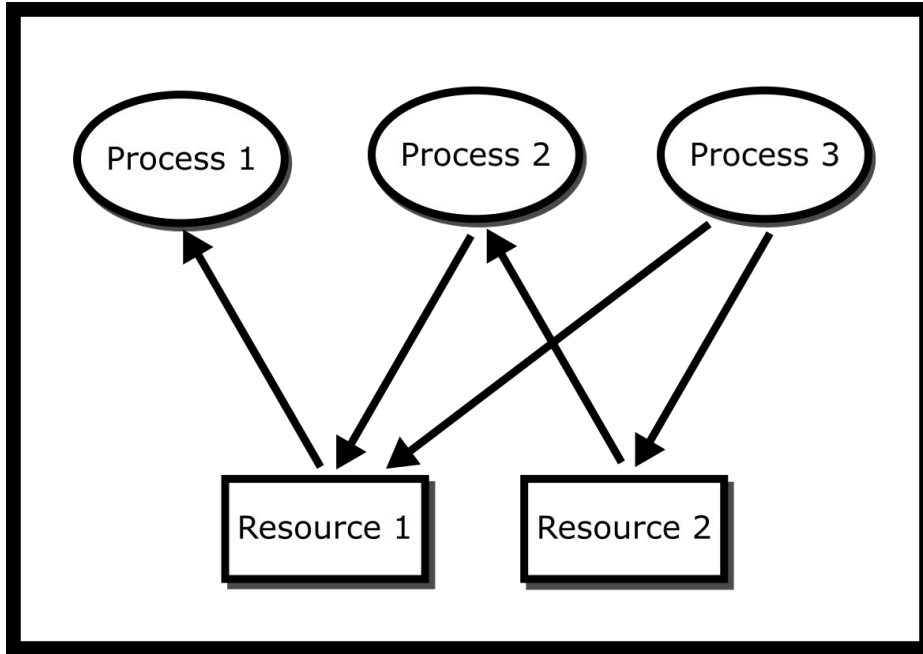
Livelock

A thread often acts in response to the action of another thread. If the other thread's action is also a response to the action of another thread, then *livelock* may result.

Starvation

Starvation describes a situation where a thread is unable to gain regular access to shared resources and is unable to make progress.

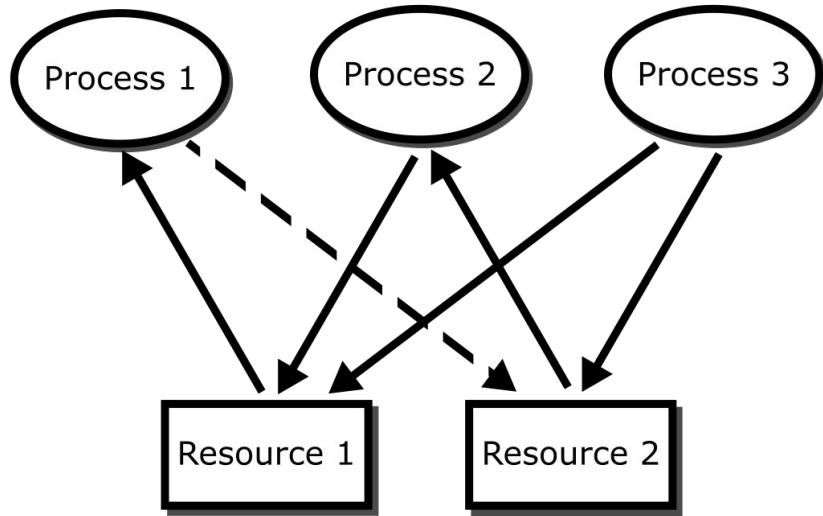
Deadlock: Resource allocation graphs



If there is a cycle in the Resource Allocation Graph and each resource in the cycle provides only one instance, then the processes will deadlock.

We can detect a deadlock by traversing the graph and searching for a cycle using a graph traversal algorithm, such as the Depth First Search (DFS).

Deadlock: Resource allocation graphs



If there is a cycle in the Resource Allocation Graph and each resource in the cycle provides only one instance, then the processes will deadlock.

We can detect a deadlock by traversing the graph and searching for a cycle using a graph traversal algorithm, such as the Depth First Search (DFS).

Coffman conditions

Deadlock can happen if and only if the four Coffman conditions are satisfied.

1. **Mutual Exclusion:** No two processes can obtain a resource at the same time.
2. **Circular Wait:** There exists a cycle in the Resource Allocation Graph:
 - a. there exists a set of processes $\{P_1, P_2, \dots\}$ such that P_1 is waiting for resources held by P_2 , which is waiting for $P_3, \dots$, which is waiting for P_1 .
3. **Hold and Wait:** Once a resource is obtained, a process keeps the resource locked.
4. **No Pre-emption:** Nothing can force the process to give up a resource.

Proof in the book

See also simulations in

https://en.wikipedia.org/wiki/Deadlock#Individually_necessary_and_jointly_sufficient_conditions_for_deadlock

Approaches to solve deadlock

Ostrich algorithm:

ignore the deadlock, prevent processes from context-switching

"to stick one's head in the sand and pretend there is no problem".

Used in most operating systems

Banker's algorithm

https://en.wikipedia.org/wiki/Banker%27s_algorithm

- How much of each resource each process could possibly request ("MAX")
- How much of each resource each process is currently holding ("ALLOCATED")
- How much of each resource the system currently has available ("AVAILABLE")

Dining philosopher problem

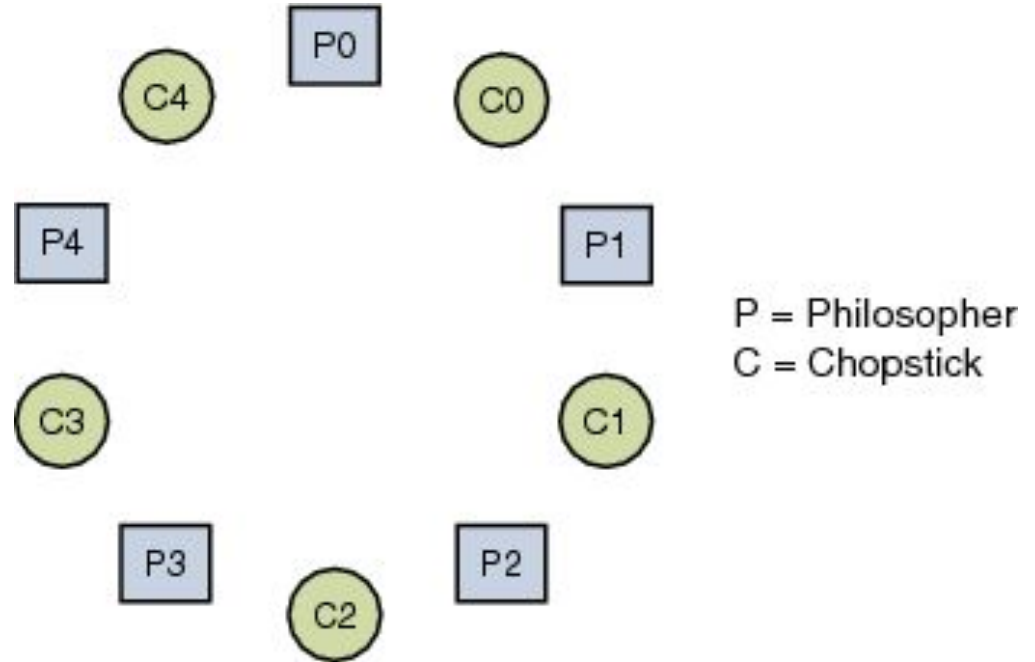
Five philosophers, numbered zero to four, are sitting at a round table, thinking.

As time passes, different individuals become hungry and decide to eat.

There is a platter of noodles on the table but each philosopher only has one chopstick to use.

In order to eat, they must share chopsticks.

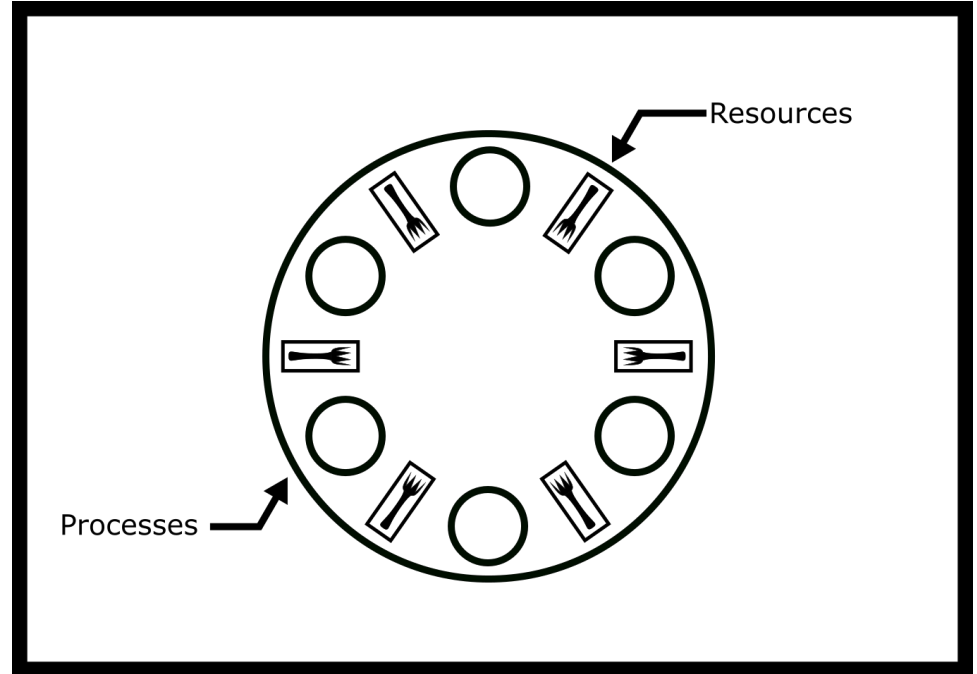
The chopstick to the left of each philosopher (as they sit facing the table) has the same number as that philosopher.



- think until the left fork is available; when it is, pick it up
- think until the right fork is available; when it is, pick it up
- eat for a fixed amount of time
- put the right fork down
- put the left fork down
- think for a fixed amount of time
- repeat

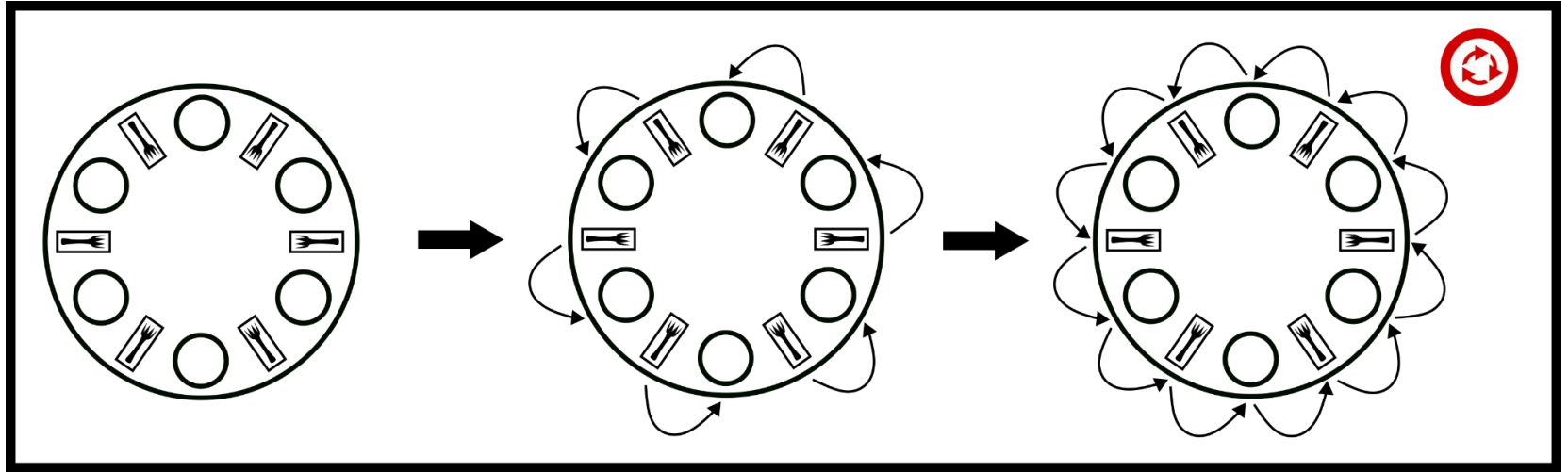
Dining philosopher problem

```
philosopher(){  
    while(true){  
        think()  
        pick(left_chopstick)  
        pick(right_chopstick)  
        eat()  
        putdown_chopsticks()  
    }  
}
```



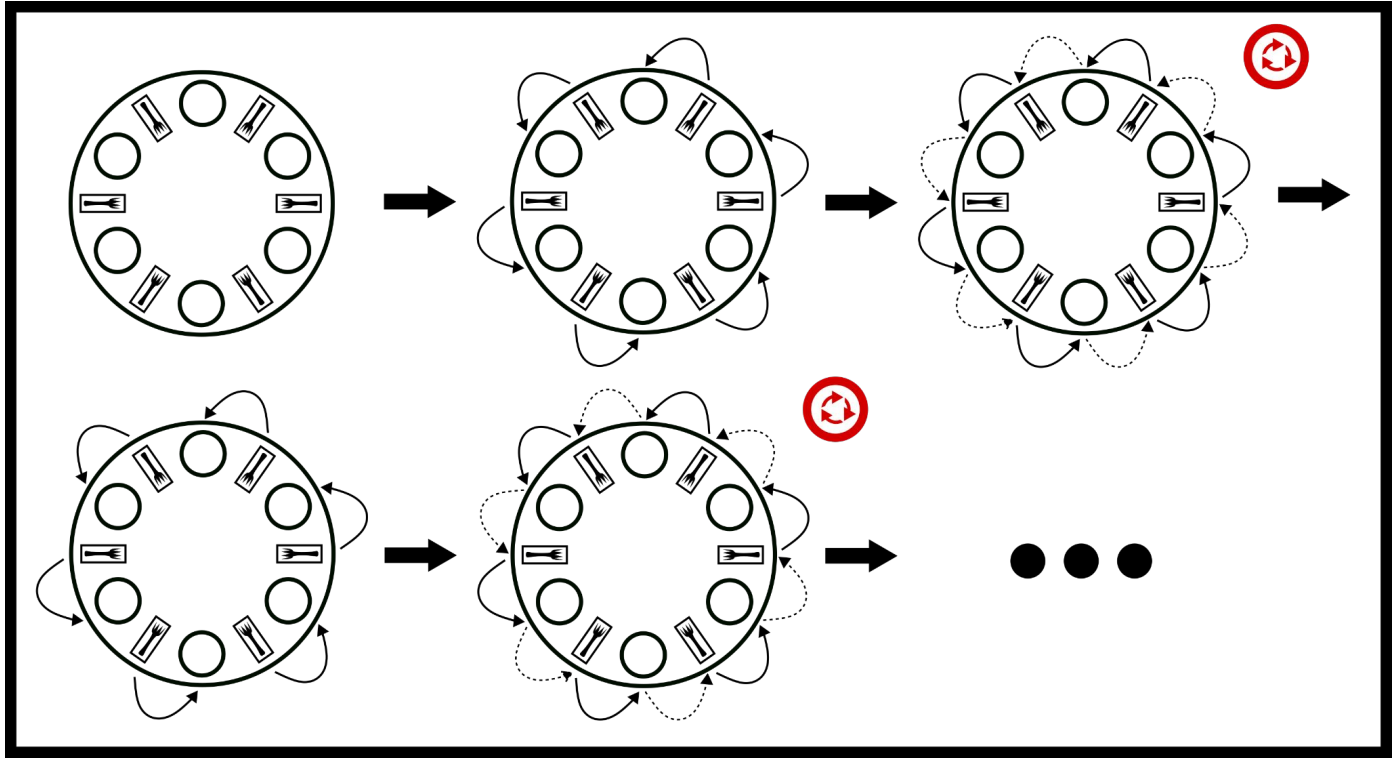
How deadlock occurs?

What if everyone picks up their left chopstick and is waiting on their right chopstick?



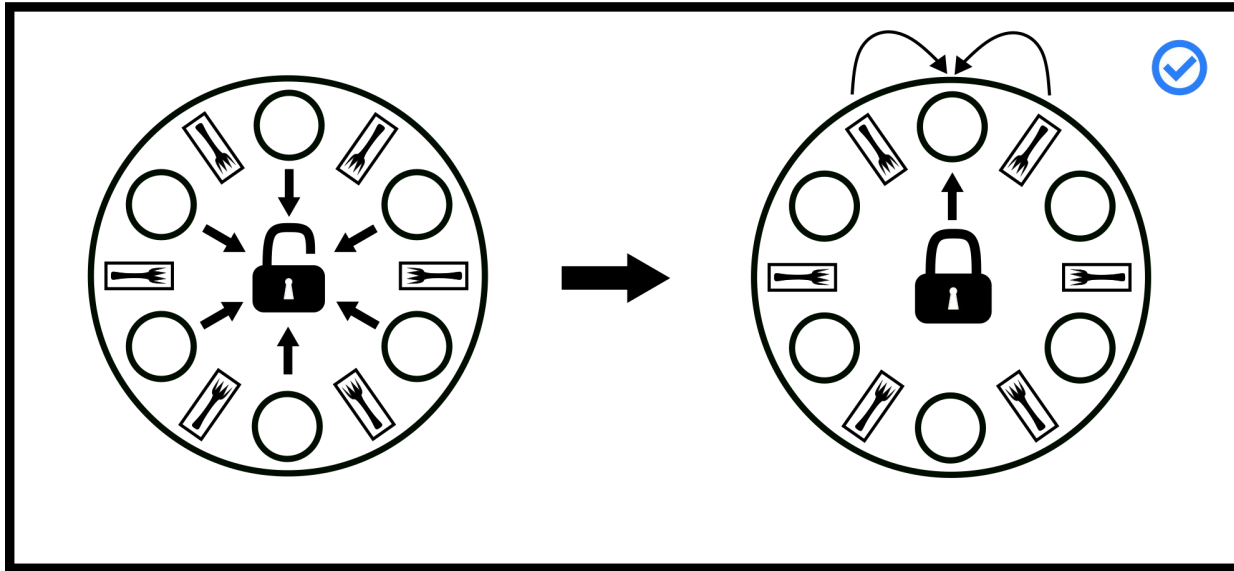
resource allocation graph that shows how the system could be deadlocked

LIVELOCK: a time evolution of the break hold and wait would look like.



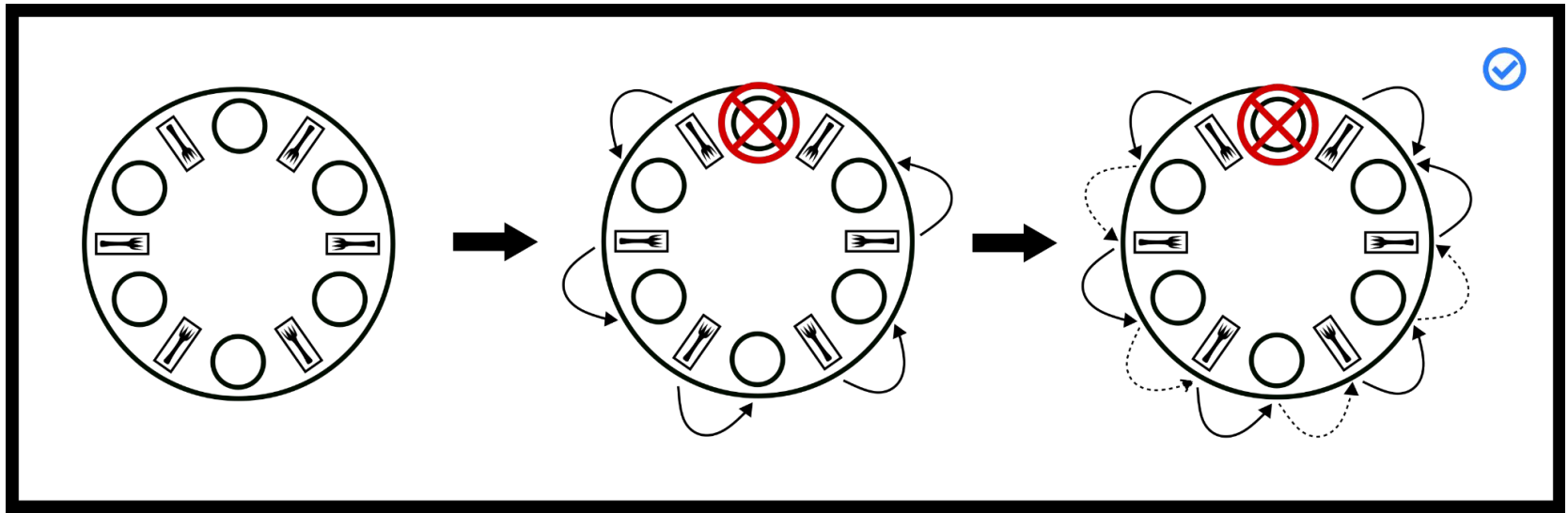
Arbitrator solution

- A philosopher can only pick up both or none by introducing an arbitrator, e.g., a waiter
 - Waiter can be implemented as a mutex
- Works but parallelism is reduced
 - All philosophers wait for waiter



Stalling solution (leaving table)

Remove philosophers until deadlock is not possible. No circular wait means deadlock is not possible.

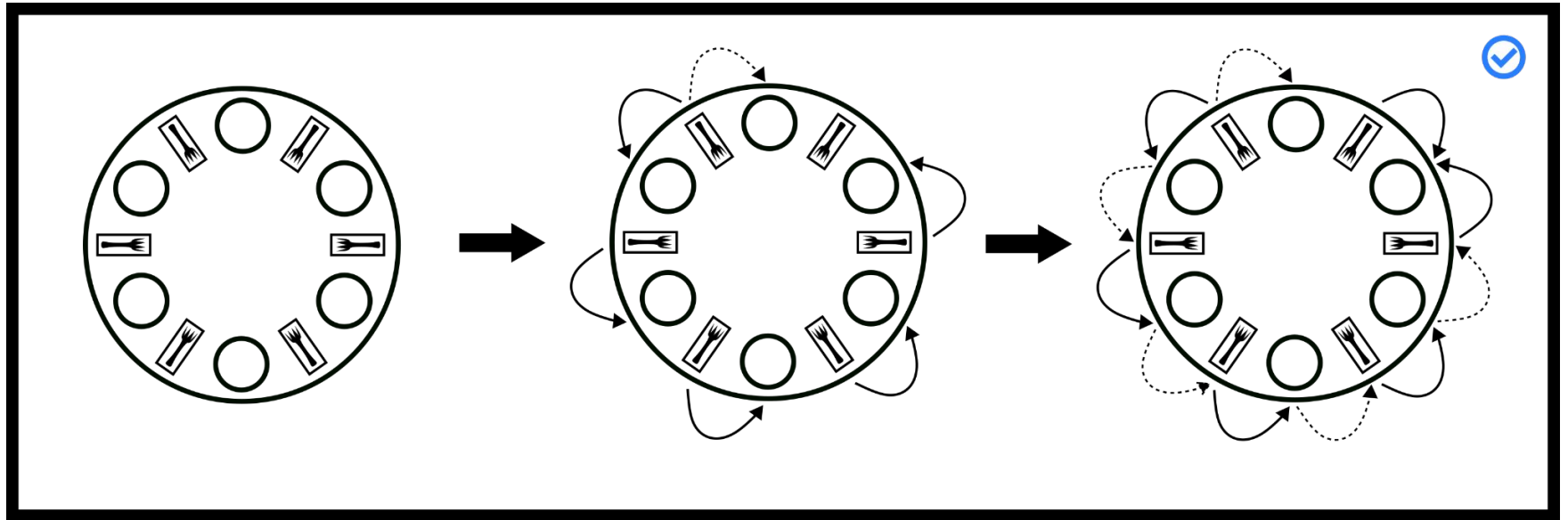


Partial (hierarchical) Ordering (Dijkstra's solution)

- Assign a number to each philosopher and chopstick
- A philosopher can choose two chopsticks
 - i ,
 - $i+1 \% 5$
- Each philosopher takes **the lower numbered chopstick first**
 - P0-C0,
 - P2-C2,
 - P3-C3,
 - **P4-cannot take C0**
 - The highest numbered philosopher does not have any chopstick.
- Then each gets the higher numbered chopstick
 - **P4 is able to get C5 and eat**

Partial (hierarchical) ordering (Dijkstra's solution)

Each process picks only lower numbered chopstick first, then the higher numbered chopstick.



Barriers: next week

Suppose we wanted to perform a multi-threaded calculation that has two stages, but we don't want to advance to the second stage until the first stage is completed.

We could use a synchronization method called a **barrier**.

When a thread reaches a **barrier**, it will wait at the barrier until all the threads reach the barrier, and then they'll all proceed together.

Pthreads has a function **pthread_barrier_wait()** that implements this.

You'll need to declare a **pthread_barrier_t** variable

and initialize it with **pthread_barrier_init()**. **pthread_barrier_init()** takes the number of threads that will be participating in the barrier as an argument.